🎮 Autonomous Survival Game in C Programming

A Game Development Mini-Project Built Entirely in C Programming Language

Presented By:

- Anuraj S Davange - 3745
- Aditya Ekbote - 0270
- Mann S. Chouhan - 0671
- Vedant Borse - 1604
- Manas Lanke - 3276

Roles:

- Analyzer
- Generator
- Generator
- Testing & Debugging
- Testing & Debugging

MIT-WPU | FOP | Kajal Vatekar Ma'am | 13/02/2026

C-Powered Game of Survival

Mastering C Programming Through Gaming

1

Pure C Console Survival Game

Developed entirely using the C programming language, this game runs directly in the console, showcasing C's fundamental capabilities.

2

C-Implemented Autonomous Player

The player character automatically navigates the map to collect food items, driven by C-coded navigation logic.

3

Dynamic Enemy AI in C

Enemies intelligently chase the player, implemented with efficient algorithms entirely in C, adding a layer of challenge and strategy.

4

Real-Time C Simulation

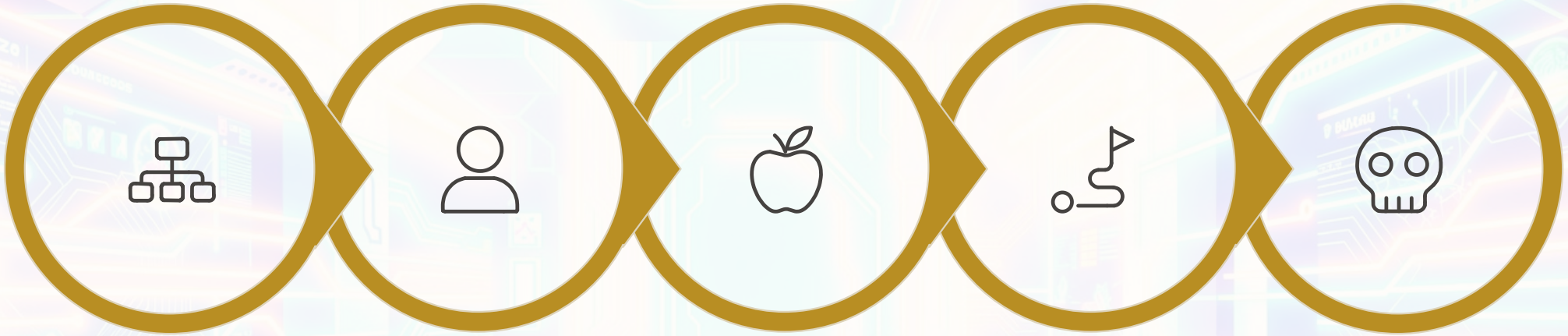
Experience a fast-paced game simulation without relying on external game engines, demonstrating the raw power and efficiency of C programming.

Objective:

- Apply core C programming concepts to a practical, engaging gaming project.
- Implement an efficient pathfinding algorithm for intelligent character movement using C.
- Develop a robust real-time game system from scratch in C.

Made with GAMMA

How the C-Language Game Works



Map Gen

Player Spawn

Food Appears

Pathfinding

Enemy Move

This console-based survival game, built entirely with the C programming language, simulates a dynamic environment where strategic movement and quick reflexes are key to survival. It showcases robust C-language implementation for real-time game mechanics.

Made with GAMMA

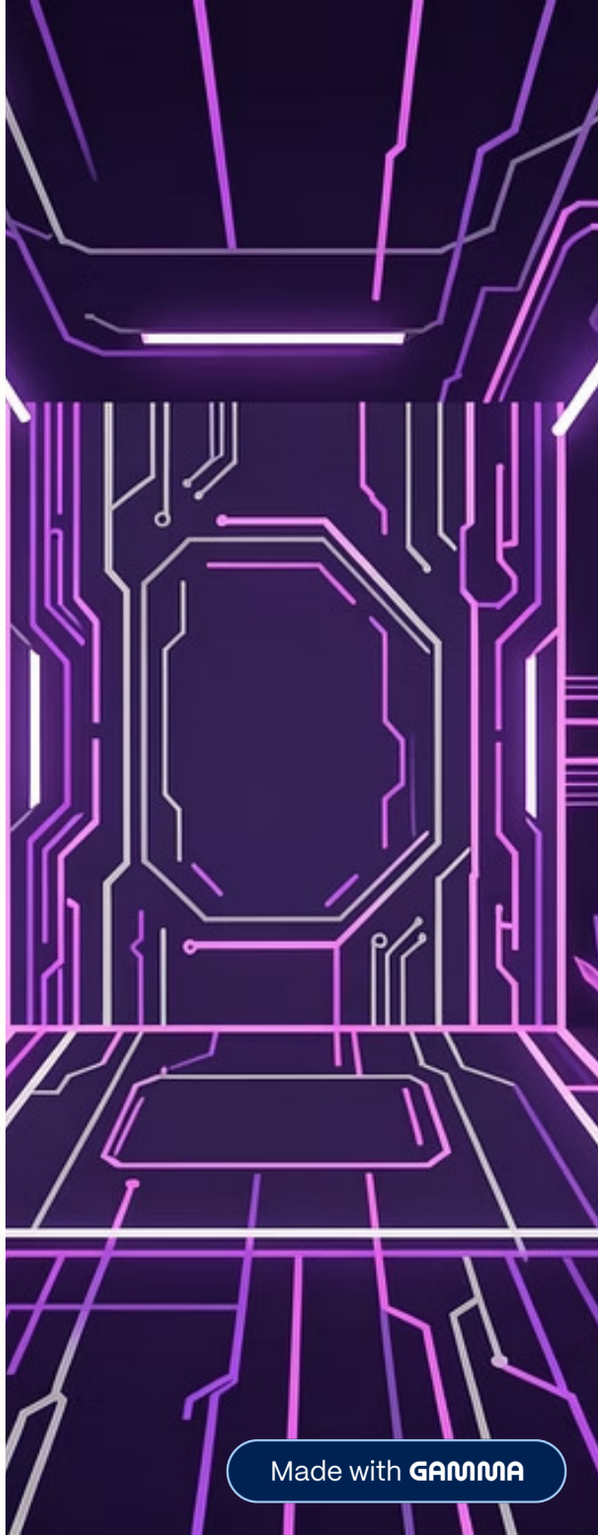
C Language Gaming: Technologies & Core Concepts

Key Technologies for our C Game

- **C Programming Language:** The robust foundation for our entire game engine.
- **C Structures & Arrays:** Fundamental for efficient game data and state management.
- **BFS (Breadth First Search) Algorithm:** Implemented in C for efficient in-game pathfinding.
- **OpenMP (C Parallel Processing):** Optimizing game performance for real-time computations.
- **Windows API / ncurses:** Enabling C-based console graphics and cross-platform input for our game.

Core Game Development Concepts in C

- **C Pathfinding:** Crafting intelligent navigation algorithms directly in C.
- **The C Game Loop:** Orchestrating real-time updates and rendering within our C engine.
- **C Collision Detection:** Precisely managing game entity interactions in C.
- **C Real-time Updates:** Delivering a responsive and dynamic game environment entirely in C.



Made with GAMMA

C Language Game Features

Automatic Player Movement in C

The player autonomously seeks out and collects food using an intelligent pathfinding algorithm, coded entirely in C.

C-Implemented Enemy Chase Logic

Enemies use their own C-implemented AI to track and pursue the player, increasing the challenge.

C-Based Shooting Mechanism

Players can fire projectiles, a core C game mechanic, to defend themselves against approaching enemies.

C-Managed Health & Score System

Track player health and accumulated score for a competitive and engaging experience, managed efficiently by C structures.

Dynamic Enemy Spawning in C

New enemies appear periodically, keeping the game challenging and unpredictable, a testament to C's performance in dynamic environments.

Cross-Platform Support with C

Designed to run on various operating systems, thanks to standard C libraries and conditional compilation, showcasing C's versatility.

Game Symbols

@	Player
M	Monster
!	Food
~	Obstacle
*	Bullet

Made with GAMMA

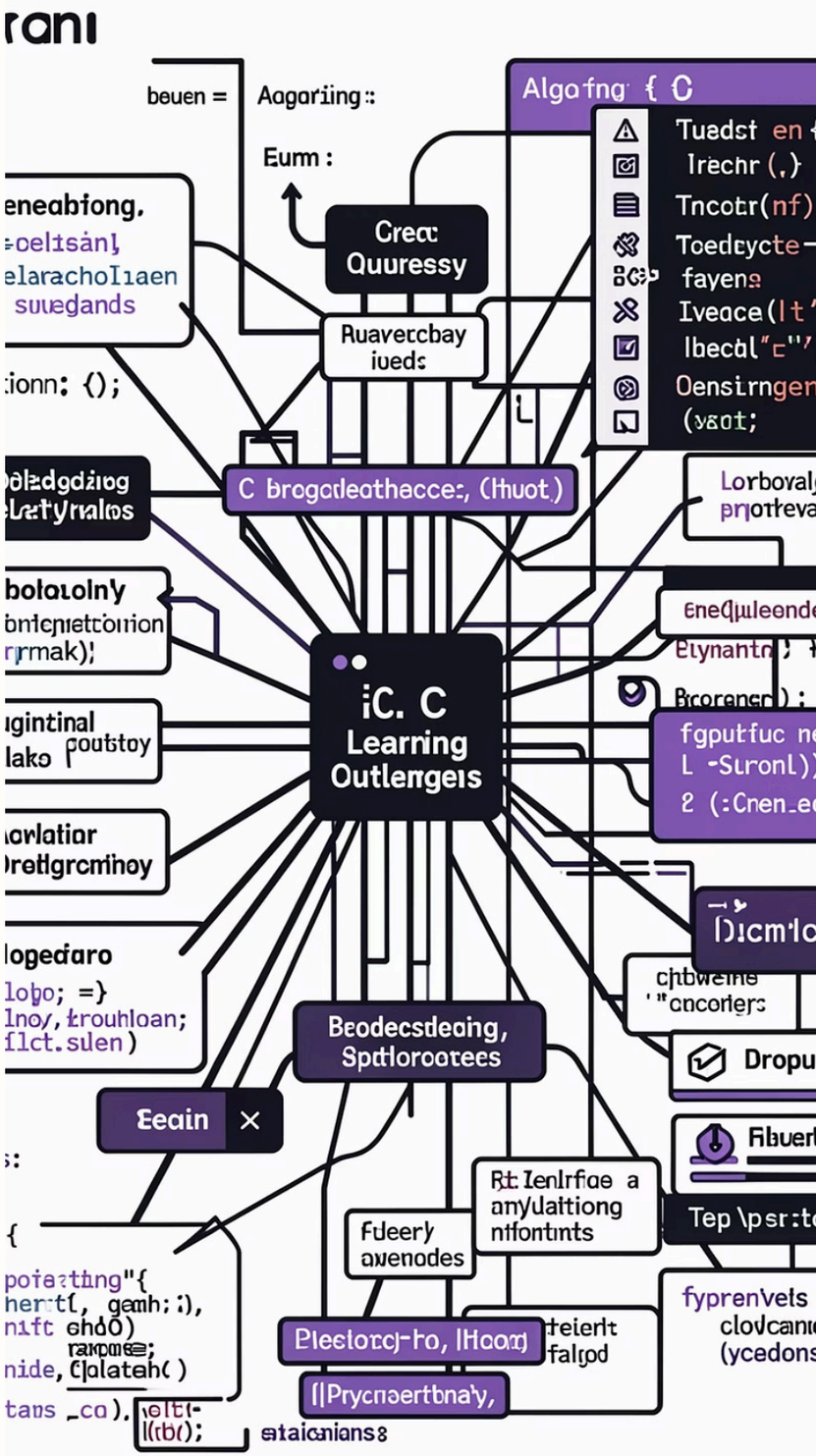
Challenges & Learning in C Game Development

Challenges Faced

- **Implementing BFS Pathfinding for a C-based game:** Optimising the algorithm for efficiency in a real-time environment.
- **Managing Real-Time Updates in a C game engine:** Synchronising game states and user input for smooth gameplay.
- **Handling Collisions within the C game engine:** Accurate detection and resolution of interactions between various game objects.
- **Debugging Errors in C:** Tracing and fixing bugs in a complex C codebase, a significant learning curve in game development.

What We Learned

- **Mastery of C Programming for Games:** Deepened our knowledge of pointers, memory management, and low-level system interactions critical for game development.
- **Use of Data Structures in C:** Practical application of linked lists, queues, and arrays for game logic in a C environment.
- **Algorithm Implementation using C:** Gained hands-on experience in implementing and optimising algorithms like BFS.
- **Team Collaboration on a C game project:** Developed essential teamwork skills, including version control and task distribution.



Made with GAMMA

Conclusion & Future Scope

Conclusion

> Our project uniquely demonstrates how **pure C programming** delivers interactive, real-time gaming experiences. We've effectively implemented complex features like pathfinding and robust game mechanics, showcasing the power of **fundamental C principles** in game development.

Future Improvements

- **Keyboard Controls**
- **Better Enemy Behaviour.**
- **Sound Effects**
- **Graphical Interface Version**
- **Implementing HPA* heuristics Or Using a deque for GOAP etc**

Made with GAMMA